Mini Project – Image Processing and Computer Vision

S.No	Contents
1	Title
2	Aim
3	Objective
4	Technologies Used
5	Algorithm
6	Code
7	Features
8	Output
9	Applications
10	Limitations
11	Future Enhancements
12	Conclusion

Title :

VisionANPR Pro: Intelligent Automatic Number Plate Recognition System using OpenCV and EasyOCR

Aim :

To develop a Python-based application that detects vehicle number plates from images and extracts the plate number using image processing and Optical Character Recognition (OCR) techniques, along with logging, analytics, and a user-friendly interface.

Objective :

- Detect number plates using computer vision techniques
- Extract text using OCR
- Identify vehicle state from plate prefix
- Maintain scan logs and detect duplicates
- Provide an interactive UI using Gradio

Technologies Used :

- Python
- OpenCV (Image Processing)
- EasyOCR (Text Recognition)
- Gradio (UI Interface)
- NumPy, PIL
- CSV (Data Logging)

Algorithm :

Step 1: Image Input

- Upload vehicle image via UI

Step 2: Pre-processing

- Convert to grayscale
- Apply bilateral filter (noise reduction)
- Edge detection using Canny

Step 3: Plate Detection

- Method 1: Contour detection (4-sided polygon)
- Method 2: Morphological fallback (Top-hat + threshold)

Step 4: Plate Extraction

- Mask detected region
- Crop plate area

Step 5: Image Enhancement

- Increase contrast
- Sharpen image
- Adjust brightness

Step 6: OCR Processing

- Apply thresholding
- Run EasyOCR
- Extract highest confidence text

Step 7: Post-processing

- Clean text (remove symbols)
- Convert to uppercase

Step 8: State Identification

- Extract first 2 letters

- Match with Indian state database

Step 9: Duplicate Detection

- Check if plate already scanned

Step 10: Logging

- Store results in CSV file
- Display recent scans

Step 11: Output Display

- Show annotated image
- Display plate, confidence, state
- Show statistics and logs

Code Sections include:

1. Installation
2. Imports
3. OCR Initialization
4. State Lookup
5. Logging System
6. Image Enhancement
7. Plate Detection
8. OCR Extraction
9. Gradio UI
10. Launch

CELL 1 — Install

```
!pip install -q opencv-python-headless easyocr imutils gradio pillow
```

```
# CELL 2 — Imports
import cv2
import numpy as np
import imutils
import easyocr
import gradio as gr
import csv
import os
import re
from datetime import datetime
from PIL import Image, ImageEnhance, ImageFilter
# CELL 3 — EasyOCR init
reader = easyocr.Reader(['en'], gpu=False)
# CELL 4 — Indian State/UT lookup from plate prefix
INDIA_STATE_CODES = {
    "AN":"Andaman & Nicobar","AP":"Andhra Pradesh","AR":"Arunachal Pradesh",
    "AS":"Assam","BR":"Bihar","CH":"Chandigarh","CG":"Chhattisgarh",
    "DD":"Daman & Diu","DL":"Delhi","DN":"Dadra & Nagar Haveli",
    "GA":"Goa","GJ":"Gujarat","HP":"Himachal Pradesh","HR":"Haryana",
    "JH":"Jharkhand","JK":"Jammu & Kashmir","KA":"Karnataka","KL":"Kerala",
    "LA":"Ladakh","LD":"Lakshadweep","MH":"Maharashtra","ML":"Meghalaya",
    "MN":"Manipur","MP":"Madhya Pradesh","MZ":"Mizoram","NL":"Nagaland",
    "OD":"Odisha","PB":"Punjab","PY":"Puducherry","RJ":"Rajasthan",
    "SK":"Sikkim","TN":"Tamil Nadu","TR":"Tripura","TS":"Telangana",
```

```
    "UK":"Uttarakhand","UP":"Uttar Pradesh","WB":"West Bengal",
}

def lookup_state(plate: str) -> str:
    """Extract 2-letter state code from Indian plate and return state name."""
    if not plate:
        return "—"
    code = plate[:2].upper()
    return INDIA_STATE_CODES.get(code, "Unknown / Non-Indian")

# CELL 5 — Session log (in-memory + CSV)
LOG_FILE   = "anpr_session_log.csv"
scan_log   = []          # list of dicts for this session
seen_plates = {}         # plate -> first-seen timestamp (duplicate detection)

def init_csv():
    if not os.path.exists(LOG_FILE):
        with open(LOG_FILE, "w", newline="") as f:
            w = csv.writer(f)
            w.writerow(["#", "Plate", "Confidence", "State", "Date", "Time", "Status",
"Duplicate"])

def append_csv(entry: dict):
    with open(LOG_FILE, "a", newline="") as f:
        w = csv.writer(f)
```

```
        w.writerow([
            entry["num"], entry["plate"], entry["confidence"],
            entry["state"], entry["date"], entry["time"],
            entry["status"], entry["duplicate"]
        ])

init_csv()

def build_log_html() -> str:
    """Render last 8 log entries as a styled HTML table."""
    if not scan_log:
        return """
        <div style="text-align:center;padding:28px 12px;
                    font-family:'Share Tech Mono',monospace;
                    font-size:10px;color:#3a4f65;letter-spacing:0.1em;line-height:2">
          NO RECORDS YET<br>INITIATE A SCAN TO BEGIN
        </div>"""
    rows = ""
    for e in reversed(scan_log[-8:]):
        ok      = e["status"] == "DETECTED"
        dup     = e["duplicate"] == "YES"
        col     = "#00d4ff" if ok else "#ff3939"
        dup_tag = '<span style="color:#ffb800;margin-left:6px">⚠ DUP</span>' if dup
else ""
        rows += f"""
```

```
    <tr>
      <td style="color:#3a4f65;padding:6px 8px">{e['num']}</td>
      <td style="color:{col};font-weight:600;padding:6px 8px;letter-spacing:0.1em">
       {e['plate'] or '—'}{dup_tag}
      </td>
      <td style="color:#6a84a0;padding:6px 8px">{e['confidence']}</td>
      <td style="color:#6a84a0;padding:6px 8px;font-size:9px">{e['state']}</td>
      <td style="color:#3a4f65;padding:6px 8px;font-size:9px">{e['time']}</td>
      <td style="padding:6px 8px">
       <span style="color:{'#39ff14' if ok else '#ff3939'};font-size:9px">
        {'✅' if ok else '❌'}
       </span>
      </td>
    </tr>"""
return f"""
<table style="width:100%;border-collapse:collapse;
              font-family:'Share Tech Mono',monospace;font-size:10px">
  <thead>
    <tr style="border-bottom:1px solid #1f2d42">
      <th style="color:#3a4f65;padding:4px 8px;text-align:left">#</th>
      <th style="color:#3a4f65;padding:4px 8px;text-align:left">PLATE</th>
      <th style="color:#3a4f65;padding:4px 8px;text-align:left">CONF%</th>
      <th style="color:#3a4f65;padding:4px 8px;text-align:left">STATE</th>
      <th style="color:#3a4f65;padding:4px 8px;text-align:left">TIME</th>
```

```
        <th style="color:#3a4f65;padding:4px 8px;text-align:left">OK</th>
      </tr>
    </thead>
    <tbody>{rows}</tbody>
  </table>"""

def build_stats_html() -> str:
  total   = len(scan_log)
  ok      = sum(1 for e in scan_log if e["status"] == "DETECTED")
  fail    = total - ok
  dups    = sum(1 for e in scan_log if e["duplicate"] == "YES")
  rate    = f"{round(ok/total*100)}%" if total else "—"
  avg_conf = (
    f"{round(sum(float(e['confidence'].strip('%')) for e in scan_log if e['confidence'] != '—') / max(ok,1))}%"
    if ok else "—"
  )
  def card(num, label, color="#00d4ff"):
    return f"""
    <div style="background:#0e1420;border:1px solid #1f2d42;border-radius:4px;
                padding:12px 14px;position:relative;overflow:hidden">
      <div style="position:absolute;top:0;left:0;right:0;height:1px;
                  background:linear-gradient(90deg,transparent,{color},transparent);opacity:0.4"></div>
      <div style="font-family:'Share Tech Mono',monospace;font-size:26px;
```

```
                color:{color};line-height:1">{num}</div>
        <div style="font-family:'Share Tech Mono',monospace;font-size:8px;
                color:#3a4f65;letter-spacing:0.2em;margin-top:4px">{label}</div>
      </div>"""
  return f"""
  <div style="display:grid;grid-template-columns:repeat(3,1fr);gap:8px;margin-bottom:10px">
    {card(total,"TOTAL SCANS")}
    {card(ok,"DETECTED","#39ff14")}
    {card(fail,"FAILED","#ff3939")}
  </div>
  <div style="display:grid;grid-template-columns:repeat(3,1fr);gap:8px">
    {card(rate,"SUCCESS RATE","#ffb800")}
    {card(avg_conf,"AVG CONFIDENCE")}
    {card(dups,"DUPLICATES","#ffb800")}
  </div>"""

# CELL 6 — Image enhancement helpers
def enhance_image(img_bgr: np.ndarray) -> np.ndarray:
  """Boost contrast + sharpen to help OCR on poor-quality images."""
  pil = Image.fromarray(cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB))
  pil = ImageEnhance.Contrast(pil).enhance(1.6)
  pil = ImageEnhance.Sharpness(pil).enhance(2.0)
  pil = ImageEnhance.Brightness(pil).enhance(1.1)
  return cv2.cvtColor(np.array(pil), cv2.COLOR_RGB2BGR)
```

```
def deskew_plate(crop: np.ndarray) -> np.ndarray:
    """Attempt to straighten a slightly tilted plate crop."""
    coords = np.column_stack(np.where(crop > 0))
    if coords.shape[0] == 0:
        return crop
    angle = cv2.minAreaRect(coords)[-1]
    if angle < -45:
        angle = 90 + angle
    if abs(angle) < 1:
        return crop
    h, w = crop.shape[:2]
    M = cv2.getRotationMatrix2D((w // 2, h // 2), angle, 1.0)
    return cv2.warpAffine(crop, M, (w, h), flags=cv2.INTER_CUBIC,
                          borderMode=cv2.BORDER_REPLICATE)

# CELL 7 — Core detection (multi-method with fallback)
def find_plate_contour(gray: np.ndarray, edged: np.ndarray):
    """Primary method: find largest 4-point contour."""
    keypoints = cv2.findContours(
        edged.copy(), cv2.RETR_TREE, cv2.CHAIN_APPROX_SIMPLE
    )
    contours = imutils.grab_contours(keypoints)
    contours = sorted(contours, key=cv2.contourArea, reverse=True)[:15]
```

```
    for cnt in contours:
        peri  = cv2.arcLength(cnt, True)
        approx = cv2.approxPolyDP(cnt, 0.02 * peri, True)
        if len(approx) == 4:
            x, y, w, h = cv2.boundingRect(approx)
            ratio = w / float(h)
            if 1.5 < ratio < 6.0:       # realistic plate aspect ratio
                return approx
    return None


def find_plate_morphology(gray: np.ndarray):
    """Fallback method: morphological top-hat + threshold."""
    kernel   = cv2.getStructuringElement(cv2.MORPH_RECT, (13, 5))
    tophat   = cv2.morphologyEx(gray, cv2.MORPH_TOPHAT, kernel)
    _, thresh = cv2.threshold(tophat, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
    thresh   = cv2.dilate(thresh, kernel, iterations=2)
    cnts, _  = cv2.findContours(thresh, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    cnts     = sorted(cnts, key=cv2.contourArea, reverse=True)[:5]
    for cnt in cnts:
        x, y, w, h = cv2.boundingRect(cnt)
        ratio = w / float(h)
        if 1.5 < ratio < 6.0 and w > 60:
            pts = np.array([[x,y],[x+w,y],[x+w,y+h],[x,y+h]], dtype=np.int32)
            return pts.reshape(4, 1, 2)
```

```
    return None


def run_ocr_on_crop(crop: np.ndarray):
    """Run EasyOCR with preprocessing; return (text, confidence_pct_str)."""
    # Resize small crops for better OCR
    h, w = crop.shape[:2]
    if w < 200:
        scale = 200 / w
        crop  = cv2.resize(crop, (int(w*scale), int(h*scale)), interpolation=cv2.INTER_CUBIC)

    # Try deskew
    crop = deskew_plate(crop)

    # Denoise + threshold for cleaner characters
    crop = cv2.fastNlMeansDenoising(crop, h=10)
    _, crop_bin = cv2.threshold(crop, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

    results = reader.readtext(crop_bin, detail=1, paragraph=False)
    if not results:
        results = reader.readtext(crop, detail=1, paragraph=False)
    if not results:
        return "", "—"

    # Highest confidence result
```

```
    best      = sorted(results, key=lambda r: r[2], reverse=True)[0]
    raw_text  = best[1]
    conf      = round(best[2] * 100)
    clean     = "".join(c for c in raw_text if c.isalnum()).upper()
    return clean, f"{conf}%"


def detect_plate(image, enhance: bool):
    """
    Full ANPR pipeline with fallback detection + enhancement.
    Returns: annotated_img, plate, confidence, state, date, time, status, log_html,
stats_html, csv_path
    """
    if image is None:
        return None, "", "—", "—", "", "", "❌ No image uploaded", build_log_html(),
build_stats_html(), None


    now  = datetime.now()
    date = now.strftime("%Y-%m-%d")
    time = now.strftime("%H:%M:%S")


    # ── Pre-processing ──────────────────────────────────────────────────────
    img = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    if enhance:
        img = enhance_image(img)
```

```
gray    = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
bfilter = cv2.bilateralFilter(gray, 11, 17, 17)
edged   = cv2.Canny(bfilter, 30, 200)

# ── Plate detection (primary → fallback) ──────────────────────
location = find_plate_contour(gray, edged)
method   = "CONTOUR"
if location is None:
    location = find_plate_morphology(gray)
    method   = "MORPHOLOGY"

if location is None:
    entry = {
        "num": len(scan_log) + 1, "plate": "", "confidence": "—",
        "state": "—", "date": date, "time": time,
        "status": "FAILED", "duplicate": "NO"
    }
    scan_log.append(entry)
    append_csv(entry)
    return (
        image, "", "—", "—", date, time,
        "❌ No plate region found — try a clearer image",
        build_log_html(), build_stats_html(), LOG_FILE
    )
```

```
    # ── Crop + OCR ──────────────────────────────────────────────────────────
    mask = np.zeros(gray.shape, np.uint8)
    cv2.drawContours(mask, [location], 0, 255, -1)
    (xs, ys) = np.where(mask == 255)
    x1, y1  = int(np.min(xs)), int(np.min(ys))
    x2, y2  = int(np.max(xs)), int(np.max(ys))
    cropped  = gray[x1:x2+1, y1:y2+1]

    clean_text, confidence = run_ocr_on_crop(cropped)

    # ── State lookup ────────────────────────────────────────────────────────
    state = lookup_state(clean_text)

    # ── Duplicate check ─────────────────────────────────────────────────────
    is_dup = False
    if clean_text:
        if clean_text in seen_plates:
            is_dup = True
        else:
            seen_plates[clean_text] = time

    # ── Draw annotation ─────────────────────────────────────────────────────
```

```
output = img.copy()
color  = (0, 255, 70)
cv2.drawContours(output, [location], 0, color, 3)
for pt in location.reshape(4, 2):
    cv2.circle(output, tuple(pt.astype(int)), 6, (0, 255, 200), -1)

if clean_text:
    font, fs, thick = cv2.FONT_HERSHEY_SIMPLEX, 1.0, 2
    label           = f"  {clean_text}  "
    (lw, lh), base  = cv2.getTextSize(label, font, fs, thick)
    lx = int(location[0][0][0])
    ly = int(location[1][0][1]) + 54
    cv2.rectangle(output, (lx, ly-lh-base-4), (lx+lw, ly+base), (0, 18, 0), -1)
    cv2.rectangle(output, (lx, ly-lh-base-4), (lx+lw, ly+base), color, 2)
    cv2.putText(output, label, (lx, ly-base), font, fs, color, thick, cv2.LINE_AA)

    # Method tag
    tag_label = f"[{method}]  {confidence}"
    cv2.putText(output, tag_label, (lx, ly + base + 20),
                font, 0.45, (0, 200, 180), 1, cv2.LINE_AA)

if is_dup:
    cv2.putText(output, "⚠️ DUPLICATE PLATE", (10, 34),
                cv2.FONT_HERSHEY_SIMPLEX, 0.8, (255, 180, 0), 2, cv2.LINE_AA)
```

```
    output_rgb = cv2.cvtColor(output, cv2.COLOR_BGR2RGB)

    # ── Log ──────────────────────────────────────────────────────────────
    status = "DETECTED" if clean_text else "OCR_FAILED"
    entry  = {
        "num": len(scan_log) + 1, "plate": clean_text,
        "confidence": confidence, "state": state,
        "date": date, "time": time,
        "status": status, "duplicate": "YES" if is_dup else "NO"
    }
    scan_log.append(entry)
    append_csv(entry)

    dup_msg   = "  ⚠️ DUPLICATE PLATE ALERT" if is_dup else ""
    status_ui = f"✅ {clean_text} — {state}{dup_msg}" if clean_text else "⚠️ Plate region found but OCR failed"

    return (
        output_rgb, clean_text, confidence, state,
        date, time, status_ui,
        build_log_html(), build_stats_html(), LOG_FILE
    )
```

```
def clear_session():
    scan_log.clear()
    seen_plates.clear()
    if os.path.exists(LOG_FILE):
        os.remove(LOG_FILE)
    init_csv()
    return "", "—", "—", "", "", "🗑️ Session cleared", build_log_html(), build_stats_html(),
None

# CELL 8 — CSS

CSS = """
@import
url('https://fonts.googleapis.com/css2?family=Rajdhani:wght@400;600;700&family=Shar
e+Tech+Mono&family=Exo+2:wght@300;400;500;600&display=swap');
:root{
  --bg:#080b0f;--panel:#0e1420;--panel2:#111926;
  --border:#1a2535;--border2:#1f2d42;
  --cyan:#00d4ff;--green:#39ff14;--red:#ff3939;--amber:#ffb800;
  --text:#c8d8e8;--text2:#6a84a0;--text3:#3a4f65;
  --mono:'Share Tech Mono',monospace;
  --head:'Rajdhani',sans-serif;
  --body:'Exo 2',sans-serif;
}
body,.gradio-container{background:var(--bg)!important;font-family:var(--
body)!important;color:var(--text)!important}
body::before{content:'';position:fixed;inset:0;
```

```
  background:repeating-linear-gradient(0deg,transparent,transparent
2px,rgba(0,0,0,0.025) 2px,rgba(0,0,0,0.025) 4px);

  pointer-events:none;z-index:9999}

label,.svelte-1b6s6xi{font-family:var(--mono)!important;font-size:10px!important;

  letter-spacing:0.15em!important;text-transform:uppercase!important;color:var(--
text3)!important}

.image-container,.upload-container{background:var(--panel2)!important;

  border:1px solid var(--border2)!important;border-radius:4px!important}

textarea,input[type=text]{background:var(--panel2)!important;border:1px solid var(--
border2)!important;

  border-radius:3px!important;color:var(--cyan)!important;font-family:var(--
mono)!important;

  font-size:13px!important;letter-spacing:0.1em!important}

#plate-out textarea{font-size:30px!important;letter-spacing:0.28em!important;text-
align:center!important;

  border:1px solid rgba(0,212,255,0.35)!important;box-shadow:0 0 24px
rgba(0,212,255,0.1)!important}

h3{font-family:var(--mono)!important;font-size:10px!important;letter-
spacing:0.25em!important;

  text-transform:uppercase!important;color:var(--text3)!important;

  border-bottom:1px solid var(--border2)!important;padding-
bottom:6px!important;margin:4px 0 10px 0!important}

.scan-primary{background:linear-
gradient(135deg,rgba(0,212,255,0.15),rgba(0,255,204,0.1))!important;

  border:1px solid var(--cyan)!important;color:var(--cyan)!important;

  font-family:var(--head)!important;font-size:15px!important;font-weight:700!important;

  letter-spacing:0.3em!important;text-transform:uppercase!important;
```

```
  border-radius:3px!important;padding:14px 0!important;transition:all 0.15s!important}
.scan-primary:hover{background:linear-gradient(135deg,rgba(0,212,255,0.28),rgba(0,255,204,0.18))!important;
  box-shadow:0 0 28px rgba(0,212,255,0.25)!important;transform:translateY(-1px)!important}
.clear-btn{background:rgba(255,57,57,0.07)!important;border:1px solid rgba(255,57,57,0.3)!important;
  color:#ff3939!important;font-family:var(--mono)!important;font-size:11px!important;
  letter-spacing:0.15em!important;border-radius:3px!important;padding:10px 0!important}
.clear-btn:hover{background:rgba(255,57,57,0.15)!important}
input[type=checkbox]{accent-color:var(--cyan)}
::-webkit-scrollbar{width:3px}
::-webkit-scrollbar-track{background:transparent}
::-webkit-scrollbar-thumb{background:var(--border2);border-radius:3px}
"""

# CELL 9 — UI
with gr.Blocks(css=CSS, title="VisionANPR Pro") as demo:

    # ── Top bar ──────────────────────────────────────────────────────────
    gr.HTML("""
    <div style="background:#0e1420;border-bottom:1px solid #1f2d42;
                padding:13px 24px;display:flex;align-items:center;
                justify-content:space-between;position:relative">
```

```
    <div style="display:flex;align-items:center;gap:14px">
      <div style="width:32px;height:32px;background:#00d4ff;
                  clip-path:polygon(50% 0%,100% 25%,100% 75%,50% 100%,0% 75%,0%
25%);
                  display:grid;place-items:center">
        <svg width="15" height="15" viewBox="0 0 24 24" fill="#000">
          <rect x="2" y="6" width="20" height="12" rx="2"/>
          <path d="M7 10h2M15 10h2M9 14h6" stroke="#0e1420"
                stroke-width="1.5" stroke-linecap="round" fill="none"/>
        </svg>
      </div>
      <span style="font-family:'Rajdhani',sans-serif;font-size:21px;
                   font-weight:700;letter-spacing:0.15em;color:#fff">VISIONANPR</span>
      <span style="font-family:'Share Tech Mono',monospace;font-size:9px;
                   color:#00d4ff;letter-spacing:0.2em">PRO v4.0</span>
      <div style="width:1px;height:22px;background:#1f2d42;margin:0 2px"></div>
      <div style="display:flex;align-items:center;gap:5px">
        <div style="width:6px;height:6px;border-radius:50%;background:#39ff14;
                    box-shadow:0 0 7px #39ff14;animation:hb 1.5s ease infinite"></div>
        <span style="font-family:'Share Tech Mono',monospace;font-size:9px;
                     color:#6a84a0;letter-spacing:0.1em">ENGINE ONLINE</span>
      </div>
    </div>
    <div style="display:flex;gap:8px">
```

```
        <span style="font-family:'Share Tech Mono',monospace;font-
size:9px;color:#3a4f65;
                    background:#0c1018;border:1px solid #1f2d42;padding:3px 10px;
                    border-radius:2px;letter-spacing:0.1em">OpenCV · EasyOCR ·
Contour+Morphology</span>
        <span style="font-family:'Share Tech Mono',monospace;font-
size:9px;color:#3a4f65;
                    background:#0c1018;border:1px solid #1f2d42;padding:3px 10px;
                    border-radius:2px;letter-spacing:0.1em">Indian Plate DB · CSV Logger ·
Dup Alert</span>
      </div>
      <div style="position:absolute;bottom:0;left:0;right:0;height:1px;
                background:linear-gradient(90deg,transparent,#00d4ff,transparent)"></div>
    </div>
    <style>@keyframes
hb{0%,100%{opacity:1;transform:scale(1)}50%{opacity:.5;transform:scale(1.3)}}</style>
    """)

    # ── Body
──────────────────────────────────────────────────────────────────────────────

    with gr.Row(equal_height=False):

        # ── LEFT col ──────────────────────────────────────────────────────────
        with gr.Column(scale=5):
            gr.Markdown("### // Input Feed")
            image_input = gr.Image(type="numpy", label="Upload Vehicle Image",
height=290)
```

```
        gr.Markdown("### // Annotated Output")
        image_output = gr.Image(label="Detection Result", height=290,
interactive=False)

    # ── RIGHT col ──────────────────────────────────────────────────────
    with gr.Column(scale=4):

        # Plate display
        gr.Markdown("### // Detected Plate")
        plate_out = gr.Textbox(
            label="Plate Number", placeholder="——————————",
            interactive=False, elem_id="plate-out"
        )

        # Confidence + State row
        with gr.Row():
            conf_out  = gr.Textbox(label="Confidence",  interactive=False, scale=1)
            state_out = gr.Textbox(label="State / Region", interactive=False, scale=2)

        # Date / Time row
        with gr.Row():
            date_out = gr.Textbox(label="Date", interactive=False, scale=1)
            time_out = gr.Textbox(label="Time", interactive=False, scale=1)
```

```
        # Status
        status_out = gr.Textbox(label="Status", interactive=False)

        # Options
        with gr.Row():
            enhance_toggle = gr.Checkbox(
                label="🔬 Image Enhancement (contrast + sharpen)",
                value=True
            )

        # Buttons
        with gr.Row():
            scan_btn  = gr.Button("⬡  INITIATE SCAN", variant="primary",
                                  elem_classes=["scan-primary"], scale=3)
            clear_btn = gr.Button("✕  CLEAR", elem_classes=["clear-btn"], scale=1)

        # CSV download
        gr.Markdown("### // Export Log")
        csv_out = gr.File(label="Download Session CSV", interactive=False)

# ── Bottom — Stats + Log ──────────────────────────────────────────────
with gr.Row():
    with gr.Column(scale=1):
        gr.Markdown("### // Session Statistics")
```

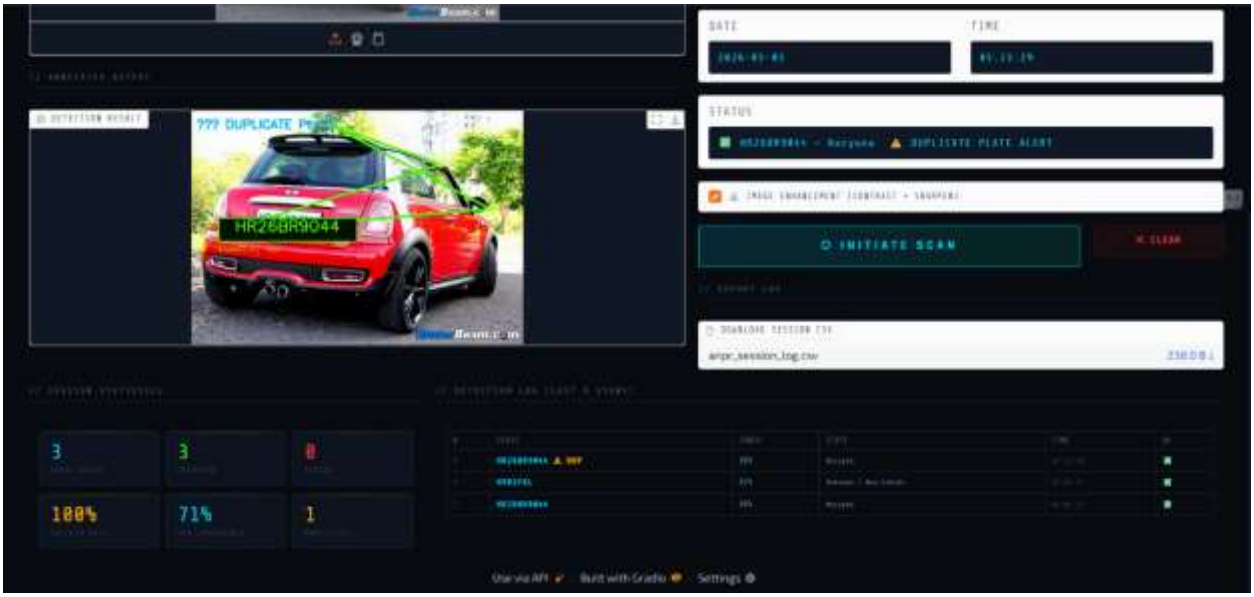
```
        stats_html = gr.HTML(build_stats_html())

    with gr.Column(scale=2):
        gr.Markdown("### // Detection Log  (last 8 scans)")
        log_html = gr.HTML(build_log_html())

# ── Wire ──────────────────────────────────────────────────────────────────

scan_btn.click(
    fn=detect_plate,
    inputs=[image_input, enhance_toggle],
    outputs=[image_output, plate_out, conf_out, state_out,
             date_out, time_out, status_out,
             log_html, stats_html, csv_out]
)

clear_btn.click(
    fn=clear_session,
    inputs=[],
    outputs=[plate_out, conf_out, state_out,
             date_out, time_out, status_out,
             log_html, stats_html, csv_out]
)
# CELL 10 — Launch
demo.launch(share=True)
```

Features

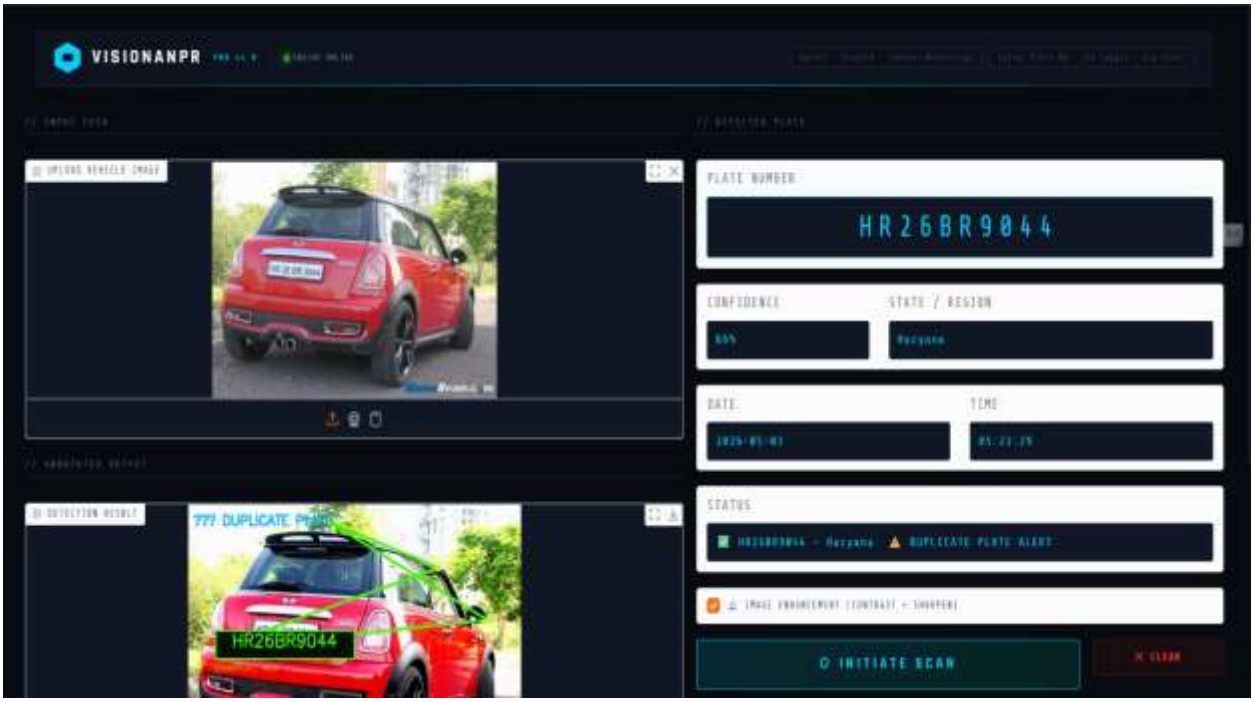
- Multi-method plate detection
- OCR with confidence score
- Image enhancement option
- Indian state identification
- Duplicate plate alert
- CSV log export
- Real-time statistics dashboard
- Professional dark UI

Output

- Detected number plate displayed
- Confidence percentage
- State/Region identified
- Annotated image with bounding box
- Session statistics and logs
- Downloadable CSV report



Output Screen 1



Output Screen 2

Applications

- Traffic monitoring systems
- Smart parking systems
- Toll collection automation
- Law enforcement & surveillance
- Vehicle tracking systems

Limitations

- Performance depends on image quality
- Low accuracy for blurred/angled plates
- OCR errors in complex backgrounds
- Works best with standard plate formats

Future Enhancements

- Real-time video detection (CCTV integration)
- Deep learning-based detection (YOLO)
- Multi-language OCR
- Cloud database integration
- Mobile app version

Conclusion

The VisionANPR Pro system successfully demonstrates how computer vision and OCR can be integrated to automate number plate recognition. It provides accurate detection, real-time logging, and an interactive interface, making it suitable for real-world intelligent transport systems.